



LazyHC2L Developer & Team Guide



Overview



Objective

The LazyHC2L tool extends the **Hierarchical Cut 2-Hop Labeling (HC2L)** algorithm to handle **dynamic and disruption-prone road networks** using two mechanisms:

- **Lazy Updates:** Defer recomputation until affected nodes are queried.
- **Threshold-Based Triggering:** Only update when disruption impact exceeds a defined severity threshold (τ).

The goal is to deliver **efficient real-time routing** that balances accuracy and computation cost while adapting to real-world road changes (traffic, closures, accidents).



System Architecture Summary



Current Architecture Components (from Thesis)

1. **Dataset Input** – imports Quezon City OSM road network + HERE Traffic Flow data.
2. **Network Change Detector** – identifies disrupted edges.
3. **Impact Evaluator** – computes $\text{ImpactScore} = f(\Delta w) \times f(\text{jam}) \times f(\text{closure})$
4. **Threshold Decision Module** – compares $\text{ImpactScore} \geq \tau$ to decide update type.
5. **Lazy Update Mode** – marks nodes as dirty, updates on query.
6. **Immediate Update Mode** – recomputes all affected labels instantly.
7. **Query Engine** – processes source-destination requests, handles lazy repair.
8. **Frontend (Flask + JS)** – displays route, traffic, disruptions, and comparisons.

Algorithm Summary

Component	Description
HC2L	Static hierarchical labeling algorithm with fast query and small label size.
DHL (Baseline)	Dynamic labeling using dual hierarchies for query/update.
LazyHC2L (Proposed)	Adaptive HC2L with lazy and threshold-based label repair.

Lazy Update Logic

- Labels are **not recomputed immediately** when a disruption occurs.
- Affected nodes are marked **dirty** until a query touches them.
- When queried, the algorithm **repairs** only the relevant subgraph.

Threshold Triggering Logic

- Computes disruption severity via **ImpactScore**.
- If **ImpactScore** $\geq \tau$ $\rightarrow$ immediate recompute.
- If **ImpactScore** $< \tau$ $\rightarrow$ lazy mark only.

PANEL FEEDBACK $\rightarrow$ ACTION PLAN

Panel Concern	Required Fix / Feature	Implementation Area
Real-world adaptation	Integrate real APIs (MMDA, Waze, HERE); simulate disruptions dynamically	Backend & Simulation
Algorithm clarity	Highlight that HC2L is <i>labeling-based</i> , not a shortest path algorithm	UI & Docs
Performance stress test	Add performance/memory dashboard	Backend & UI

Visualization gap	Visualize DHL vs LazyHC2L updates on map	Frontend (Map overlay)
Simulation realism	Add time-based traffic profiles, rerouting logic	Simulation Layer
User input	Allow search-based locations, not only pin	Frontend (Geolocation)
Accuracy validation	Compare with Google Maps (Fréchet + Overlap)	Evaluation Module
Memory optimization	Add LRU caching, label pruning	Backend Core
One-way roads	Enforce OSM directionality	Graph Preprocessing

USER INTERFACE DEVELOPMENT TASKS

A. Input & Interaction

Component	Description	Status	Action
 Search Bar	Text-based location search (Nominatim / Geopy)	✗	Add
 Smart Pinning	Pins snap to nearest road	⚠	Fix
 Disruption Reporter	User can mark closure/accident	✓	Improve with type icons
✗ Unreport / Delete	Remove disruption pins	✗	Add
 Reroute Button	Recalculate after disruption	✗	Add
 Time Profile Selector	Morning / Noon / Evening	✗	Add

B. Map & Visualization

Component	Function	Action
-----------	----------	--------

 Path Lines	Color-coded routes (LazyHC2L, DHL, Google)	Polish colors
 Traffic Overlay	Visualize congestion level	Add Layer
 Update Region Overlay	DHL vs LazyHC2L recomputed areas	Add via Leaflet polygons
 Legend Panel	Standardized colors (Blue=Lazy, Purple=DHL)	Add
 Route Info Panel	Show ETA, label count, algorithm	Enhance
 Export Results	Save JSON/CSV logs for thesis appendix	Add

C. Legend & Dashboard

Component	Description	Status
 Comparison Dashboard	Side-by-side DHL vs LazyHC2L	Add
 Metrics Chart	Label time, size, query response	Add
 Live Log Panel	Per-query debug logs	Add
 SOP Mode Toggle	Strict mode for thesis data capture	Optional

D. Simulation Controls

Feature	Description	Action
 Run Simulation	Start scenario (e.g., Scenario 1–5)	Update flow
 Reset Simulation	Clear all pins/routes	Improve
 Load Scenarios	Predefined or random disruptions	Add
 Live/Step Toggle	Replay algorithm updates	Optional

DEVELOPER / BACKEND TASKS

Core Algorithm Enhancements

1. Lazy Update System

- Add `dirty_labels` set and `mark_as_dirty()` logic.
- Implement `lazy_update()` trigger on query.
- Log: `ImpactScore`, `UpdateType`, and node IDs affected.

2. Threshold Logic Integration

- Compute $\text{ImpactScore} = f\Delta w \times f_{jam} \times f_{closure}$.
- Compare to τ and trigger Lazy or Immediate Update.
- Allow τ slider in frontend for demo.

3. Disruption Handling API

- `/add_disruption`, `/clear_disruption`, `/reroute` endpoints.
- Handle static + simulated inputs.

4. Performance Optimization

- Add **LRU cache** for reused paths.
- Implement **garbage collection** for stale labels.
- Record memory/time logs.

5. Graph Directionality Fix

- Use `network_type="drive"` in OSMnx to enforce one-way.
- Validate edges using `oneway` tag.

6. Logging & Visualization Output

- Output updated node lists for both DHL and LazyHC2L.

Return JSON to frontend:

```
{
  "disruption_segment": [104,105],
  "dhl_updated_nodes": [1,2,3,4,5],
  "lazy_updated_nodes": [3,4]
}
```

○

SIMULATION & TESTING REQUIREMENTS

Test Scenarios

Scenari	Description
○	
1	Light congestion only
2	Road closure
3	Mixed (accident + congestion)
4	Randomized disruptions
5	Historical rush-hour simulation

Metrics to Log

- Labeling time (s) - How long it takes to build the label index for the graph using HC2L or its variants. This includes preprocessing steps like graph partitioning, label construction, and pruning.
 - Used in: SOP 1 & 2 – System efficiency
 - Interpreted as: Lower = faster system initialization
- Labeling size (MB) - Total memory size of the constructed labels after preprocessing. Represents the storage required for all node labels in the index.
 - Used in: SOP 1 & 2
 - Interpreted as: Lower = more efficient memory use
- Query response (μs) - Time taken to compute a single shortest path between an origin and destination using the label index. For LazyHC2L, this includes any triggered lazy update time.

- Used in: SOP 1 & 2
- Interpreted as: Lower = faster pathfinding at runtime

- Fréchet distance (m) - Measures the geometric similarity between two paths (e.g., LazyHC2L vs Google Maps). Lower Fréchet values mean the paths are spatially closer in shape and alignment.
 - Used in: SOP 3 – Accuracy
 - Interpreted as: Lower = route is more similar to reference

- Segment overlap (%) - Percentage of road segments shared between the tool-generated route and the reference route (Google Maps). Calculated by matching IDs or spatial proximity of segments.
 - Used in: SOP 3
 - Interpreted as: Higher = route is closer to reference path

- Nodes updated per algorithm - Total number of nodes that had their labels recomputed after a disruption.
- For DHL: updates are immediate and broad
- For LazyHC2L: updates are lazy and limited (triggered on query)
 - Used in: SOP 2 – System responsiveness
 - Interpreted as: Lower = more efficient adaptation

- Memory used (MB) - Total runtime memory consumed during simulation, including label storage, dirty flag states, and temporary buffers.
 - Used in: SOP 1 + optimization testing
 - Interpreted as: Lower = better scalability

Optional Formatting Tip (for Tooltips or UI Panels)

Metric	Tooltip
Labeling Time	"Time to construct all node labels before queries"
Labeling Size	"Total memory occupied by all precomputed labels"

Query Response	"Time taken to compute shortest path using labels"
Fréchet Distance	"Geometric difference between tool route and reference"
Segment Overlap	"How many road segments match the reference path"
Nodes Updated	"Number of nodes reprocessed after disruption"
Memory Used	"Overall memory usage during simulation run"

Output Format

CSV / JSON export for each trial (for SOP evaluation). Include:

trial,algorithm,label_time,label_size,query_time,frechet,overlap,updated_nodes



DELIVERABLES FOR PROGRAMMER

Deliverable	Description
✓ Working Backend	Fully functional LazyHC2L + DHL comparison engine
✓ UI Dashboard	User + Developer views with overlays & metrics
✓ Simulation Suite	Scenario loader, rerouting demo
✓ Export System	JSON/CSV logs for thesis results
✓ Documentation	Inline code comments + API endpoints list



Suggested Dev Tools & Frameworks

- **Backend:** C++, Flask (Python), SQLite, NumPy, Pandas
 - **Frontend:** Leaflet.js / Mapbox GL JS, Bootstrap / Material UI
 - **Visualization:** Chart.js / Plotly for dashboard
 - **Testing:** pytest (Python), CSV output validation
-



Implementation Flow Summary

1. Load base map (Quezon City, OSMnx)
 2. Load disruptions (HERE or synthetic)
 3. Run DHL and LazyHC2L
 4. Log updated nodes & query times
 5. Display overlay comparison
 6. Export results (for SOPs)
-



Success Criteria

- Clear visual proof of LazyHC2L's adaptive update vs DHL.
- Correct handling of one-way and restricted roads.
- Consistent comparison results with Google Maps.
- Logged metrics usable for SOP 1–3 analysis.
- Frontend simulation runs smoothly and interactively.

SAMPLE UI ONLY

User ViewDeveloper View

LabelsUpdatesDebug

Update Scope Comparison

DHL Update1,243 nodes

Full recomputation triggered immediately

LazyHC2L Update87 nodes

Lazy update on query only

Threshold Trigger

Impact Score:0.68

Threshold:0.50

Update triggered on next query

Dirty Labels

Marked as dirty (not yet recomputed):

Nodes: 45, 67, 83, 102, 134

Will update when query hits these nodes

Timeline

t=0ms: Disruption detected

t=5ms: Labels marked dirty

t=150ms: Query received

t=187ms: Lazy update completed

Map Visualization Area

Routes, disruptions, and traffic overlays appear here

Disruption ahead

reroute?

YesNo

User ViewDeveloper View

LabelsUpdatesDebug

Playback Controls

Reset

Play

Next

Graph Snapshot

Before Disruption

After Update

Label Source Log

[INFO] Path Found via Label Intersection

[DEBUG] S(12) → H(83) → D(198)

[INFO] Label(S) = [45, 83, 102, 156]

[INFO] Label(D) = [83, 134, 198]

[SUCCESS] Common hub: 83

[DEBUG] Distance: 12.4km via hub

[INFO] No node exploration needed

[WARN] Label 45 marked dirty

[SUCCESS] Lazy update complete

Debug Export

Export Debug Log (JSON)

Export Label Updates (CSV)

Enable SOP Mode

Locks simulation conditions for SOP 1-3 testing

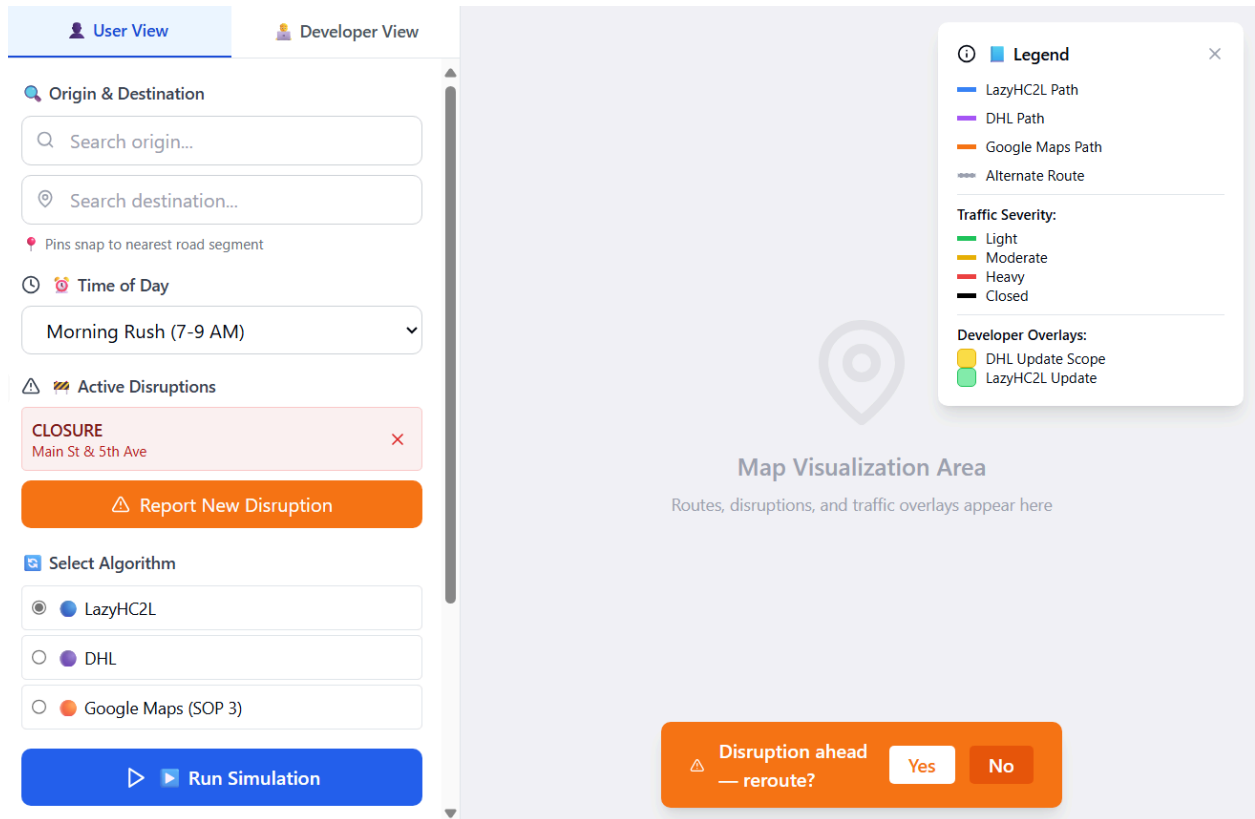
Map Visualization Area

Routes, disruptions, and traffic overlays appear here

Disruption ahead

reroute?

YesNo



Current UI:

